

In [1]: `import pandas as pd`

In [2]: `import numpy as np`

In [3]: `data=pd.read_csv(r"C:\Users\hp\Downloads\ADST.PROJECT.csv")`

In [4]: `data`

Out[4]:

	Year	Month	Region	Vehicle_Type	Model	Sales_Channel	Units_Sold	Average
0	2022	1	North America	Hybrid	Camry	Corporate	1063	
1	2019	8	Japan	Pickup	Fortuner	Fleet	968	
2	2023	12	Asia	EV	Prius	Online	2197	
3	2020	11	North America	EV	Fortuner	Fleet	954	
4	2022	12	Latin America	EV	Prius	Fleet	1406	
...	...	...	...	...	...	...	...	
995	2025	10	Japan	SUV	Fortuner	Fleet	732	
996	2025	2	North America	EV	RAV4	Corporate	628	
997	2023	2	Middle East	Hatchback	Land Cruiser	Online	1177	
998	2017	1	Japan	SUV	Fortuner	Dealer	152	
999	2024	10	Asia	Hybrid	Fortuner	Dealer	2435	

1000 rows × 15 columns



In [5]: `data.shape`

Out[5]: (1000, 15)

In [6]: `data.head()`

Out[6]:

	Year	Month	Region	Vehicle_Type	Model	Sales_Channel	Units_Sold	Average_Pi
--	------	-------	--------	--------------	-------	---------------	------------	------------

0	2022	1	North America	Hybrid	Camry	Corporate	1063	
1	2019	8	Japan	Pickup	Fortuner	Fleet	968	
2	2023	12	Asia	EV	Prius	Online	2197	
3	2020	11	North America	EV	Fortuner	Fleet	954	
4	2022	12	Latin America	EV	Prius	Fleet	1406	



In [7]: `data.tail()`

Out[7]:

	Year	Month	Region	Vehicle_Type	Model	Sales_Channel	Units_Sold	Average
--	------	-------	--------	--------------	-------	---------------	------------	---------

995	2025	10	Japan	SUV	Fortuner	Fleet	732	
996	2025	2	North America	EV	RAV4	Corporate	628	
997	2023	2	Middle East	Hatchback	Land Cruiser	Online	1177	
998	2017	1	Japan	SUV	Fortuner	Dealer	152	
999	2024	10	Asia	Hybrid	Fortuner	Dealer	2435	



In [8]: `data.info()`

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Year                                1000 non-null   int64
1   Month                              1000 non-null   int64
2   Region                             1000 non-null   str
3   Vehicle_Type                       1000 non-null   str
4   Model                              1000 non-null   str
5   Sales_Channel                      1000 non-null   str
6   Units_Sold                         1000 non-null   int64
7   Average_Price_USD                 1000 non-null   int64
8   Revenue_USD                       1000 non-null   int64
9   Cost_USD                          1000 non-null   float64
10  Operating_Expenses_USD             1000 non-null   float64
11  Profit_USD                         1000 non-null   float64
12  Profit_Margin_Percent              1000 non-null   float64
13  Market_Share_Percent               1000 non-null   float64
14  Customer_Satisfaction              1000 non-null   float64
dtypes: float64(6), int64(5), str(4)
memory usage: 143.5 KB
```

In [9]: `data.describe()`

Out[9]:

	Year	Month	Units_Sold	Average_Price_USD	Revenue_USD	Co
count	1000.000000	1000.000000	1000.000000	1000.000000	1.000000e+03	1.0000
mean	2020.448000	6.487000	1290.667000	41322.256000	5.357448e+07	3.8949
std	2.912005	3.510863	691.111798	13628.924842	3.520947e+07	2.6040
min	2016.000000	1.000000	85.000000	18060.000000	2.784406e+06	2.0034
25%	2018.000000	3.000000	678.750000	29433.000000	2.453717e+07	1.7551
50%	2020.000000	7.000000	1294.500000	41256.500000	4.721740e+07	3.4586
75%	2023.000000	10.000000	1903.500000	53342.000000	7.632987e+07	5.5627
max	2025.000000	12.000000	2498.000000	64900.000000	1.565162e+08	1.2053

In [10]: `print("Missing values per column:\n", data.isnull().sum())`

Missing values per column:

```

Year          0
Month         0
Region        0
Vehicle_Type  0
Model         0
Sales_Channel 0
Units_Sold    0
Average_Price_USD 0
Revenue_USD   0
Cost_USD      0
Operating_Expenses_USD 0
Profit_USD    0
Profit_Margin_Percent 0
Market_Share_Percent 0
Customer_Satisfaction 0
dtype: int64

```

In [11]: `print("Duplicate rows:", data.duplicated().sum())`

Duplicate rows: 0

In [12]: `data = data.drop_duplicates()`

In [13]: `print(data.isnull().sum())`

```
Year          0
Month         0
Region        0
Vehicle_Type  0
Model         0
Sales_Channel 0
Units_Sold    0
Average_Price_USD 0
Revenue_USD   0
Cost_USD      0
Operating_Expenses_USD 0
Profit_USD    0
Profit_Margin_Percent 0
Market_Share_Percent 0
Customer_Satisfaction 0
dtype: int64
```

```
In [14]: data['Date'] = pd.to_datetime(data['Year'].astype(str) + '-' + data['Month'].ast
```

```
In [15]: data = data.sort_values(by='Date').reset_index(drop=True)
```

```
In [16]: categorical_cols = ['Region', 'Vehicle_Type', 'Model', 'Sales_Channel']
```

```
In [17]: data_encoded = pd.get_dummies(data, columns=categorical_cols, drop_first=True)
```

```
In [18]: from sklearn.preprocessing import LabelEncoder
```

```
In [19]: le = LabelEncoder()
for col in categorical_cols:
    data[col + '_Encoded'] = le.fit_transform(data[col])
```

```
In [20]: from sklearn.preprocessing import StandardScaler
```

```
In [21]: num_cols = ['Units_Sold', 'Average_Price_USD', 'Revenue_USD', 'Cost_USD',
                    'Operating_Expenses_USD', 'Profit_USD', 'Profit_Margin_Percent',
                    'Market_Share_Percent', 'Customer_Satisfaction']
```

```
In [22]: scaler = StandardScaler()
data[num_cols] = scaler.fit_transform(data[num_cols])
```

```
In [23]: data.describe()
```

Out[23]:

	Year	Month	Units_Sold	Average_Price_USD	Revenue_USD	
count	1000.000000	1000.000000	1.000000e+03	1.000000e+03	1.000000e+03	1.0
mean	2020.448000	6.487000	1.270095e-16	-9.237056e-17	-4.440892e-17	1.0
min	2016.000000	1.000000	-1.745405e+00	-1.707684e+00	-1.443233e+00	-1.4
25%	2018.000000	3.000000	-8.858526e-01	-8.727912e-01	-8.251141e-01	-8.0
50%	2020.000000	7.000000	5.548911e-03	-4.827153e-03	-1.806405e-01	-1.0
75%	2023.000000	10.000000	8.871787e-01	8.823703e-01	6.466094e-01	6.0
max	2025.000000	12.000000	1.747817e+00	1.730844e+00	2.925157e+00	3.1
std	2.912005	3.510863	1.000500e+00	1.000500e+00	1.000500e+00	1.0



In [25]: `text_columns = ["Region", "Vehicle_Type", "Model", "Sales_Channel"]`

In [26]: `for column in text_columns:
 data[column] = data[column].str.strip()`

In [27]: `for column in text_columns:
 print(column, sorted(data[column].dropna().unique()))`

Region ['Asia', 'Europe', 'Japan', 'Latin America', 'Middle East', 'North America']

Vehicle_Type ['EV', 'Hatchback', 'Hybrid', 'Pickup', 'SUV', 'Sedan']

Model ['Camry', 'Corolla', 'Fortuner', 'Hilux', 'Land Cruiser', 'Prius', 'RAV4', 'Yaris']

Sales_Channel ['Corporate', 'Dealer', 'Fleet', 'Online']

In [28]: `print(data[["Month", "Customer_Satisfaction",
 "Profit_Margin_Percent", "Market_Share_Percent"]].describe())`

	Month	Customer_Satisfaction	Profit_Margin_Percent	\
count	1000.000000	1.000000e+03	1.000000e+03	
mean	6.487000	1.792344e-15	5.639933e-17	
std	3.510863	1.000500e+00	1.000500e+00	
min	1.000000	-3.591979e+00	-2.241819e+00	
25%	3.000000	-6.775485e-01	-8.009167e-01	
50%	7.000000	3.411465e-02	-8.510522e-03	
75%	10.000000	6.780004e-01	8.260948e-01	
max	12.000000	2.056142e+00	2.136793e+00	

	Market_Share_Percent
count	1.000000e+03
mean	4.636291e-16
std	1.000500e+00
min	-2.997701e+00
25%	-6.934446e-01
50%	1.391291e-02
75%	6.710769e-01
max	3.508873e+00

```
In [30]: print((data["Revenue_USD"] - data["Units_Sold"] * data["Average_Price_USD"]).des
```

count	1000.000000
mean	-0.025634
std	1.205671
min	-4.089151
25%	-0.680257
50%	0.404305
75%	0.710492
max	2.578754

dtype: float64

```
In [31]: print((data["Profit_USD"] - (
    data["Revenue_USD"] - data["Cost_USD"] - data["Operating_Expenses_USD"]
)).describe())
```

count	1.000000e+03
mean	-1.776357e-18
std	1.750297e+00
min	-2.458510e+00
25%	-1.411992e+00
50%	-3.499420e-01
75%	1.089997e+00
max	6.690481e+00

dtype: float64

```
In [ ]:
```